

Lab 0: Java, C++, Visual Studio Code and Git

24292-Object Oriented Programming

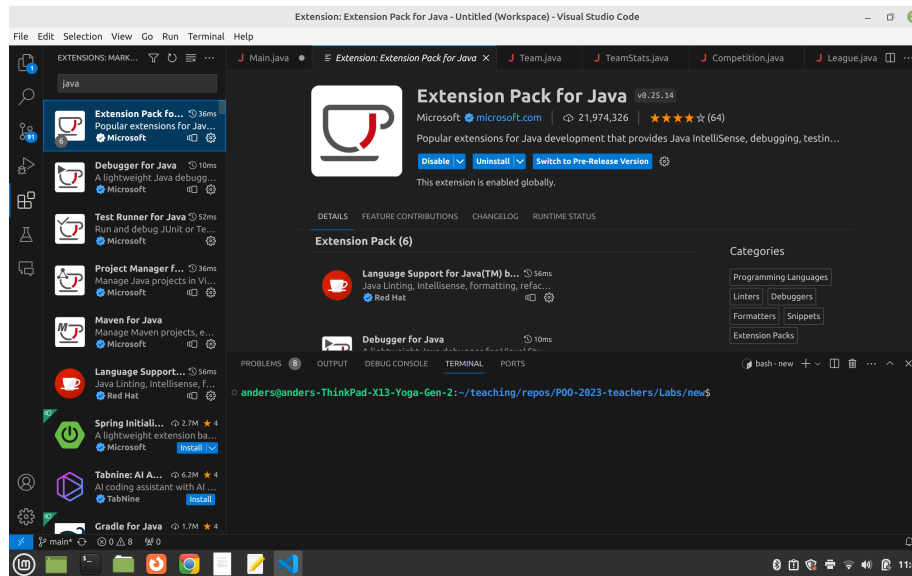
1 Introduction

The purpose of this lab is to become familiar with Java, Visual Studio Code and Git. In this lab you will implement a simple Java project, including the basic components that are always present in a Java program. Git is a version control system that allows multiple users to share and maintain projects in a central repository. The lab is introductory and will not require any delivery at the end.

2 Create a Java program

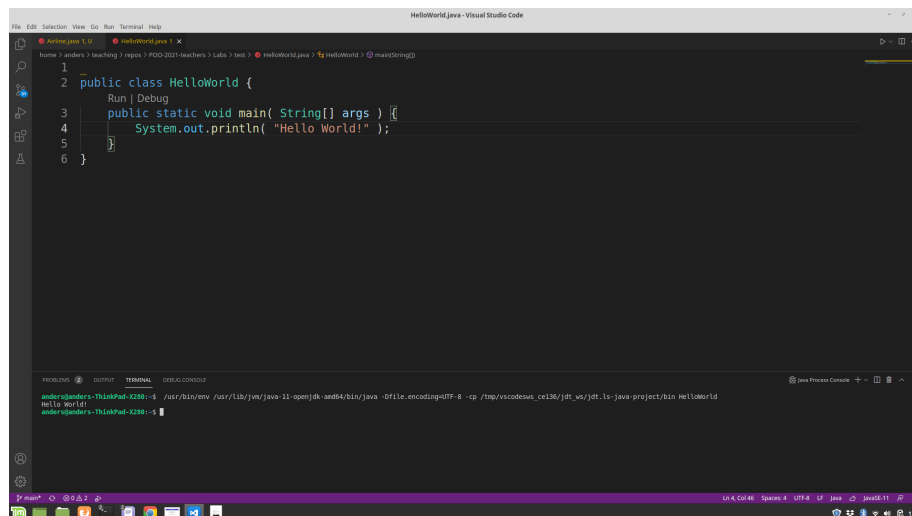
In order to implement a program in Java, it is a good idea to use an IDE (Integrated Development Environment), an editor used to develop code in various programming languages. There are many IDEs for programming in Java.

The lab handout explains how to write Java code in Visual Studio Code (VSCode). You are free to use the IDE of your choice, but the teachers may not be able to help you with the installation. To implement Java programs in VSCode, you need to 1) install the Java Development Kit (JDK); 2) install VSCode itself; and 3) install the Extension Pack for Java in VSCode (see next page). There are different versions of JDK that you can choose from (OpenJDK or Oracle), and the latest version is JDK 25.



Once the Java extension is installed you can create a new project. To ensure that VSCode finds the source code, you should **always** choose the option “Open Folder” from the menu. From the folder you can then create new files. The Java compiler requires all public classes to be stored in a file with *the same name as the class* (including uppercase and lowercase letters) and extension `.java`.

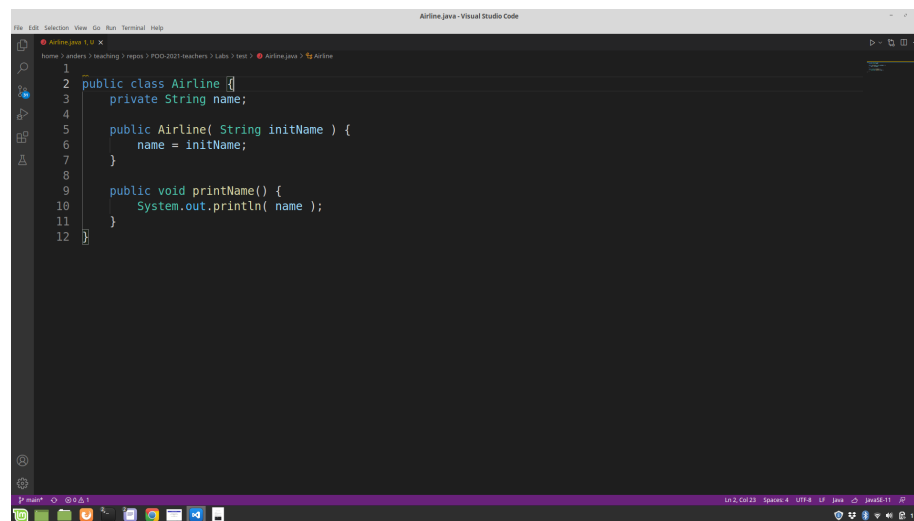
Let us define a class HelloWorld that prints “Hello World!” to the screen. The code is given below. When you run the code, a console window opens at the bottom part of the screen, and this is where the written output of the program appears. The class definition should be stored in a file “HelloWorld.java”.



From the console window it is also possible to manually compile individual Java programs and execute them from the command line. To execute Java code, we need two separate components: a *compiler* (**javac**) and an *interpreter* (**java**). The compiler takes as input Java source code and outputs an intermediate representation called byte code, stored in a file `X.class`, where `X` is the class name. The interpreter takes as input the byte code and interprets it to produce a program execution.

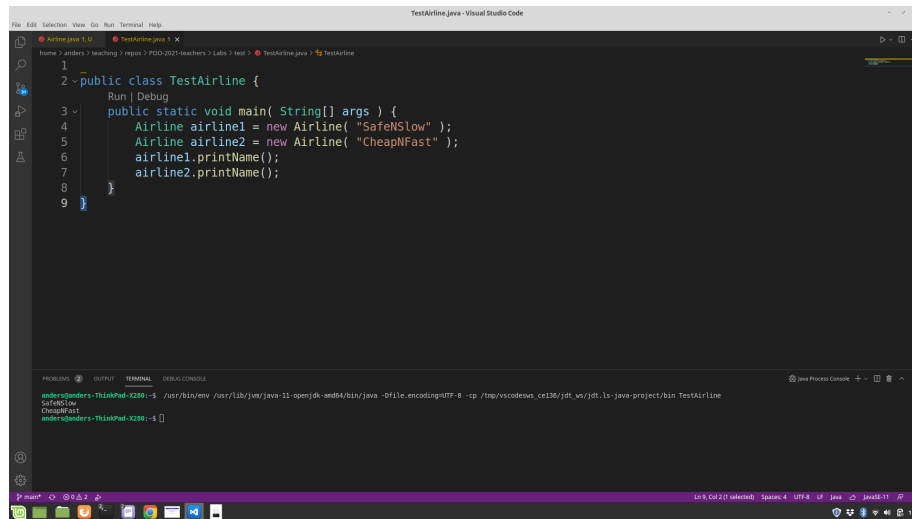
VSCoDe does not explicitly organize Java files in projects; rather, a Java project is implicitly defined by a given folder in the file system. Hence if you want multiple Java files to interact, you should always save these files in the same folder (or in subdirectories of that folder).

Next, let us define a class `Airline` with an attribute called “name”. Let us also define a constructor that takes a string argument and sets the name to the value of the argument. Finally, let us define a method called “printName” which will just print the name to the screen. Then save the class definition in a file called “`Airline.java`”. The code should look like this:



```
1 public class Airline {
2     private String name;
3
4     public Airline( String initName ) {
5         name = initName;
6     }
7
8     public void printName() {
9         System.out.println( name );
10    }
11 }
12 }
```

To execute Java code, at least one of the classes need a main method, which always has the same syntax. As a rule of thumb, it is a good idea to implement the main method in a *separate class*. In the following example, the main method is implemented in a class `TestAirline`, which tests the `Airline` class by creating instances of `Airline` and printing their names. Recall that `TestAirline` and `Airline` have to be stored in the same directory.

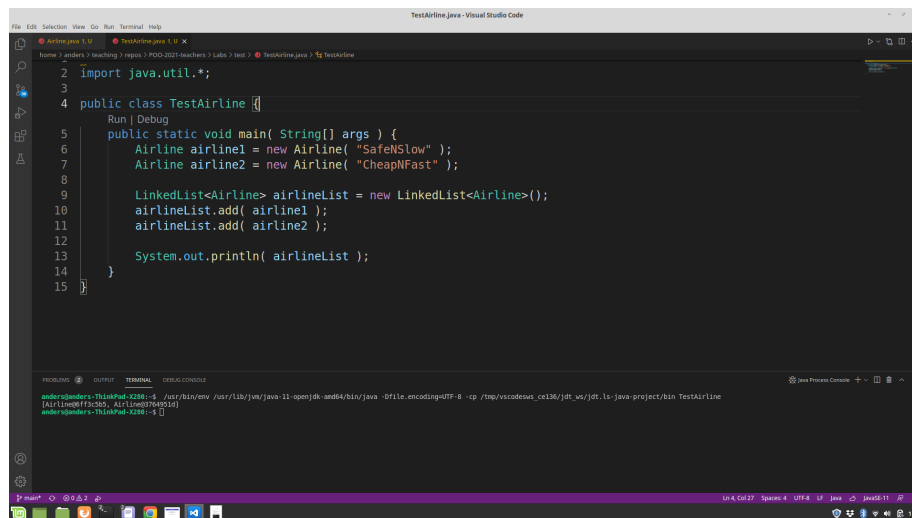


```
1 public class TestAirline {
2     public static void main( String[] args ) {
3         Airline airline1 = new Airline( "SafeNSlow" );
4         Airline airline2 = new Airline( "CheapNFast" );
5         airline1.printName();
6         airline2.printName();
7     }
8 }
9
```

```
ander@anders-ThinkPad-X280:~/4 /usr/bin/env /usr/lib/jvm/java-11-openjdk-amd64/bin/java -Dfile.encoding=UTF-8 -cp /tmp/vscodesw_cel38/jdt_wt/ls-java-project/bin TestAirline
SafeNSlow
CheapNFast
ander@anders-ThinkPad-X280:~/4
```

2.1 Trying available Java classes

Java has available a set of libraries containing lots of pre-programmed classes. It is often a good idea to reuse existing classes rather than implementing a new class from scratch. Here we include a small example of how to include a *list* of instances. More precisely, we will use a class called `LinkedList` that allows us to store any type of instance in a list data structure. To access to the `LinkedList` class, one must import it from the package `java.util` at the beginning of the source code, as in the following program:



```
2 import java.util.*;
3
4 public class TestAirline {
5     public static void main( String[] args ) {
6         Airline airline1 = new Airline( "SafeNSlow" );
7         Airline airline2 = new Airline( "CheapNFast" );
8
9         LinkedList<Airline> airlineList = new LinkedList<Airline>();
10        airlineList.add( airline1 );
11        airlineList.add( airline2 );
12
13        System.out.println( airlineList );
14    }
15 }
```

```
ander@anders-ThinkPad-X280:~/4 /usr/bin/env /usr/lib/jvm/java-11-openjdk-amd64/bin/java -Dfile.encoding=UTF-8 -cp /tmp/vscodesw_cel38/jdt_wt/ls-java-project/bin TestAirline
[Airline@7f5c85, Airline@764e51d]
ander@anders-ThinkPad-X280:~/4
```

We see that a list of instances is printed with its associated *references* (i.e. the memory address where each instance is stored). To be able to call the print

method of each instance we need to access each element in the list individually. For this purpose substitute the print line by the following loop:

```
for( Airline airline : airlineList ) {  
    airline.printName();  
}
```

Execute it and now you can see the the result correctly prints the names of each airline.

3 C++

In addition to Java, we will use C++ during the last lab of the trimester. Just like Java, there is a C/C++ extension for Visual Studio Code. In addition, you need to install a C++ compiler. For Linux and MacOS, this is extremely simple. For Windows, please follows the steps under “Configure C++ for Visual Studio Code” in the Aula Global. Please make sure that you can compile and execute C++ code before finishing the lab.

4 Git

A version control system automizes the tasks of saving, loading, registering, identifying, and mixing different versions of the files created throughout a project. In software projects, files are frequently updated, making this type of system highly useful. Files are maintained in a central repository which makes it easy to access the project remotely from different computers.

In this course we use a version control system called **Git**. Each pair of students will create and maintain their own **Git** repository, using a webpage called **GitHub** which allows free private repositories for university students. Below we explain how to use **Git** from the command line (for Linux or macOS users) or via **GitHub Desktop** (for Windows users). In **Aula Global** there is also a tutorial for setting up **Git** to work with **VSCo**. In this session, the aim is to create our first repository and test some basic **Git** commands.

4.1 Create an account on GitHub

1. Go to <https://github.com/> and create an account (one per student). It is best to use your university email since university students have free access to several premium features.
2. Log in to your account.

4.2 Create a repository

1. Choose “New” on the dashboard page to create a new repository (**only one per group**).

2. Name the repository “UPF-POO25-GX-YY” and **replace** X with a value in {101, 102, 103, 201, 202} (i.e. your lab group) and YY with a number that will be assigned to you by your lab teacher.
3. Set the access to “**Private**”.
4. Click ”Create repository”.

4.3 Cloning a repository

The next step is to clone the repository, i.e. to make a local copy of the repository on your machine. Under “Quick setup” are the instructions for cloning a repository onto your local machine. These steps are necessary each time you want to download a local copy of the repository on a new machine. After cloning the repository, create a new directory called `doc` in the repository root. Write down the answers to the questions below and save the answers to a file `answers.txt` in the `doc` directory.

4.4 Share a repository

You have to share the repository with your fellow student and your teachers (seminar and lab, plus Anders Jonsson as coordinator, i.e. up to three in total). The GitHub accounts or email addresses of your teachers are as follows:

Ahana Deb	ahanadeb
Pritam Mishra	pmishra1991
Anders Jonsson	anders.jonsson@upf.edu

In the repository, click “Settings” $\Rightarrow$ “Collaborators” $\Rightarrow$ “Add people” and enter the username or emails of the people that you want to share the repository with.

4.5 The structure of a Git project

Git projects have 3 parts that communicate with each other to save files from the local machine to the repository:

4.5.1 Working Directory

The directory in your local machine where you organize, add, delete and modify different files. Also, this is where you update the project from the repository.

4.5.2 Staging Area

The purpose of this area is to list the changes you make in your working directory, which is used to group the modifications we want to submit to the repository in a package.

4.5.3 Repository

The repository is the remote server where **Git** stores the changes from the staging area forever. Every time a change is uploaded to the repository, it is saved as a new version of the project.

4.6 Using Git

There are several ways to use Git: from the command line (Linux/macOS), from GitHub Desktop (Windows), or directly from Visual Studio Code by installing the GitHub Pull Request extension. You are free to use the Git application of your choice. In the Aula Global you will find tutorials for GitHub Desktop and for Git in Visual Studio Code. For Windows, skip to Section 3.8.

4.7 Using Git from the command line (Linux/macOS)

The following questions are for those that use Git from the **command line**.

4.7.1 Checking the status

Create a file called README.md and use the command `git status`. This file is commonly used to document the progress of the project.

Q1. What do you see on the command screen?

4.7.2 Adding a file

Now use the command `git add <file>`, and add the created file README.md.

Q2. Use the command `git status` again and explain what has changed?

4.7.3 Comparing files

Edit the README.md file adding a sentence. Now there is a difference between the README.md of your working directory, and README.md in the staging area. To see what these differences are, run the command `git diff <filename>`.

Q3. What is the output of the command? What is the current status?

Now add the README.md to the staging area.

4.7.4 First commit

All the changes must be in the staging area at this point, so the next step is to upload them to the repository, but first we need to pack everything in a new version using the command `git commit -m <message text>`. The message is used to describe the purpose of the commit. Thus, this commit will wait to be pushed to the repository as a packet, together with a version number that identifies all the previous changes.

Q4. What is a good message for the first commit? What is the current status?

4.7.5 The log information

Every time a commit is done, the log is updated, so to access the previous commits, we can run the command `git log`. In this log we save the identifier for version control, the author of the commit, the date and its message.

Q5. How many commits there are in the log? What are the first seven digits (letters and numbers) that identify the first README.md commit?

4.7.6 First push

After committed, we have to run the command `git push` that by default will upload the packet to the “origin” repository.

4.7.7 Undo Git Changes

Sometimes we make mistakes and there are two ways to recover:

1. To modify the changes and commit again.
2. To recover an earlier version of the repository.

This section will show how to erase some changes that are saved in the repository.

4.7.8 Head commit

The first thing we need is to know in which commit we are working on. The command used in this case is `git show HEAD` (HEAD always refers to the working copy).

Q6. Modify one line of the README.md, add to the staging area and commit. Then use the command `git show HEAD`, and explain the output.

4.7.9 Checkout

Remember the line you modified in the last question. You realize this line is not appropriate for the README.md file, and you want to restore the file to the one from the last commit. To do so, we use the command `git checkout HEAD <filename>`.

4.7.10 Reset

The command `git reset HEAD <filename>` is used to restore a file in the staging area and leave it equal to the file in the specified commit (in this case HEAD). This command doesn't affect to the working directory.

Q7. Rewrite the sentence you restored with the previous command. After that, add the README.md to the staging area. Then, run a reset in the README.md and explain what happens.

We can use resets in full commits by specifying the SHA (7 digits) code of the commit we want to move back of the commit history. The command is pretty much the same: `git reset <SHA>` (maybe you will need it for your project). To find the SHA of different commits, run the command `git log`.

4.8 Using GitHub Desktop (Windows)

The following questions are for GitHub Desktop in Windows.

4.8.1 Getting started

Sign in to your GitHub account. You will be asked to authorize the desktop application. Select “Clone a repository from the Internet”. You should see a list of your GitHub repositories; select the one for the OOP course. The repository should now appear in a folder “Documents/GitHub/XXX” on your computer, where “XXX” is the name of the repository.

4.8.2 Adding a new file

Create a file “README” with some text and save it in the local folder associated to your repository.

Q1. In GitHub Desktop, where does the new file appear?

To add the file to your repository, select the file and press “Commit to main”.

4.8.3 First push

The new file is now in the staging area, but it is still not part of the remote repository. To push the changes to the remote repository, select “Push commit to the origin remote”. Every time you change a file locally, it is necessary to both commit the changes and push them to the remote repository.

4.8.4 Change a file

Now change the text of the README file and save the file.

Q2. What do you see in GitHub Desktop?

Commit the new change and push the change to the remote repository.

4.8.5 Undo a change

Imagine that we want to recover the previous version of the README file.

Q3. Where can you find the history of changes to the repository?

In the History, right-click on the item that says “Create README” and select “Checkout commit”, confirming your choice.

Q4. Which version of README can you see now?

Q5. What appears in the second tab at the very top?

GitHub Desktop does not allow you to execute all Git commands. However, while in “Detached HEAD”, you can *copy* the old version of the code to another file. After doing so, in the “Detached HEAD” tab, select “Default branch → main”. This will bring back the newest version of the file. You can now copy the content of the new file back to the original file, and commit your changes. This will bring cause the old version of the file to be part of the repository again. Don’t forget to push the changes to the remote repository.

Tutorials

For a longer explanation of the commands, visit the `Git` handbook on GitHub:

<https://guides.github.com/introduction/git-handbook/>

By default, your machine uses HTTPS to communicate with the repository. For security reasons, SSH is no longer available on the university wireless networks, so we recommend everyone to use HTTPS.

Codecademy

Practice with the free `Git` course at <https://www.codecademy.com/learn/learn-git>

5 GitHub Copilot

GitHub Copilot is an extension for VSCode that allows the use of large language models (LLMs) during coding. AI and LLMs have revolutionized programming by autocompleting code as the programmer types, usually saving 50% or more of the time that the programmer spends writing an application.

We recommend that you test Copilot during the course, though be aware that you will not have access to Copilot during the exam. Copilot is free to use for verified university students, and you can find out how to become a verified student [here](#). You apply using the GitHub username (student email) that you should already have created above.